MATE SecOps Overload?
Use AI to reduce Secops noise with Mate Security.
Learn More

LARGE LANGUAGE MODELS

How to Perform Effective Agentic Context Engineering

Learn how to optimize the context of your agents, for powerful agentic performance

Eivind Kjosbakken
October 7, 2025 • 12 min read

Prefer us on Google ☆ Share

In this article, I'll discuss agentic context engineering, covering topics such as shortening the context, and tool usage. Image by ChatGPT.

Context engineering has received serious attention with the rise of LLMs capable of handling complex tasks. Initially, most discussions on this talk revolved around *prompt engineering*: Tuning a single prompt for optimized performance on a single task. However, as LLMs grow more capable, prompt engineering has turned into *context engineering*: Optimizing all data you feed into your LLM, for maximum performance on complex tasks.

In this article, I'll dive deeper into agentic context engineering, which is about optimizing the context specifically for agents. This differs from traditional context engineering in that agents typically perform sequences of tasks for a longer period of time. Since agentic context engineering is a large topic, I'll dive deeper into the topics listed below in this article and write a follow-up article covering more topics.

- Specific context engineering tips
- Shortening/summarizing the context
- Tool usage

Learn this step by step with the interactive **AI Engineer** roadmap.

Why care about agentic context engineering

AGENTIC CONTEXT ENGINEERING
WHY AGENTIC CONTEXT ENGINEERING?
LLM
?
Perform sequences of tasks
SHORTENING THE CONTEXT
Summarize important information
CONTEXT ENGINEERING TIPS
Few-shot learning
Structured prompts
Step-by-step reasoning
<A≡>
TOOL USAGE

This infographic highlights the main contents of this article. I'll first discuss why you should care about agentic context engineering. Then I'll move to specific topics within agentic context engineering, such as shortening the context, context engineering short tips, and tool usage. Image by ChatGPT.

Before diving deeper into the specifics of context engineering, I'll cover why agentic context engineering is important. I'll cover this in two parts:

1. Why we use agents
2. Why agents need context engineering

Why we use agents

First of all, we use agents because they are more capable of performing tasks that static LLM calls. Agents can receive a query from a user, for example:

Fix this user-reported bug (bug report)

This would not be feasible within a single LLM call, since you need to understand the bug better (maybe ask the person who reported the bug), you need to understand where in the code the bug occurs, and maybe fetch some of the error messages. This is where agents come in.

An agent can look at the bug, call a tool asking the user a follow-up question, for example: Where in the application does this bug occur? The agent can then find that location in the codebase, run the code itself to read error logs, and implement the fix. This all requires a series of LLM calls and tools calls before solving the issue.

Why agents need context engineering

So we now know why we need agents, but why do agents need context engineering? The main reason is that LLMs always perform better when their context contains more relevant information and less noise (irrelevant information). Furthermore, agents' context quickly adds up when they perform a series of tool calls, for example, fetching the error logs when a bug happens. This creates context bloat, which is when the context of an LLM contains a lot of irrelevant information. We need to remove this noisy information from the LLMs context, and also ensure all relevant information is present in the LLMs context.

Specific context engineering tips

Agentic context engineering builds on top of traditional context engineering. I thus include a few important points to improve your context.

- Few-shot learning
- Structured prompts
- Step-by-step reasoning

These are three commonly used techniques within context engineering that often improve LLM performance.

Few-shot learning

Few-shot learning is a commonly used approach where you include examples of a similar task before feeding the agent the task it is to perform. This helps the model understand the task better, which usually increases performance.

Below you can see two prompt examples. The first example shows a zero-shot prompt, where we directly ask the LLM the question. Considering this is a simple task, the LLM will likely get the right answer; however, few-shot learning will have a greater effect on more difficult tasks. In the second prompt, you see that I provide a few examples on how to do the math, where the examples are also wrapped in XML tags. This not only helps the model understand what task it is performing, but it also helps ensure a consistent answer format, since the model will often respond in the same format as provided in the few-shot examples.

```python
# zero-shot
prompt = "What is 123+150?"

# few-shot
prompt = """
<example>"What is 10+20?" -> "30" </example>
<example>"What is 120+70?" -> "190" </example>
What is 123+150?
"""
```

Structured prompts

Having structured prompts is also an incredibly important part of context engineering. In the code examples above, you can see me using XML tags with *<example> … </example>*. You can also use Markdown formatting to enhance the structure of your prompts. I often find that writing a general outline of my prompt first, then feeding it to an LLM for optimization and proper structuring, is a great way of designing good prompts.

You can use designated tools like Anthropic's prompt optimizer, but you can also simply feed your unstructured prompt into ChatGPT and ask it to improve your prompt. Furthermore, you'll get even better prompts if you describe scenarios where your current prompt is struggling.

For example, if you have a math agent that is doing really well in addition, subtraction, and division, but struggling with multiplication, you should add that information to your prompt optimizer.

Step-by-step reasoning

Step-by-step reasoning is another powerful context engineering approach. You prompt the LLM to think step by stepabout how to solve the problem, before attempting to solve the problem. For even better context engineering, you can combine all three approaches covered in this section, as seen in the example below:

```python
# few-shot + structured + step-by-step reasoning
prompt = """
<example>"What is 10+20?" -> "To answer the user request, I have to add up the two numb
<example>"What is 120+70?" -> "To answer the euser request, I have to add up the digits
What is 123+150?
"""
```

This will help the model understand the examples even better, which often increases model performance even further.

Shortening the context

When your agent has operated for a few steps, for example, asking for user input, fetching some information, and so on, you might experience the LLM context filling up. Before reaching the context limit and losing all tokens over this limit, you should shorten the context.

Summarization is a great way of shortening the context; however, summarization can sometimes cut out important pieces of your context. The first half of your context might not contain any useful information, while the second half includes several paragraphs that are required. This is part of why agentic context engineering is difficult.

To perform context shortening, you'll typically use another LLM, which I'll call the *Shortening LLM*. This LLM receives the context and returns a shortened version of it. The simplest version of the Shortening LLM simply summarizes the context and returns it. However, you can employ the following techniques to improve the shortening:

- Determine if some whole parts of the context can be cut out (specific documents, previous tool calls, etc)
- A prompt-tuned Shortening LLM, optimized for analyzing the task at hand, all relevant information available, and returns only the information that will be relevant to solving the task

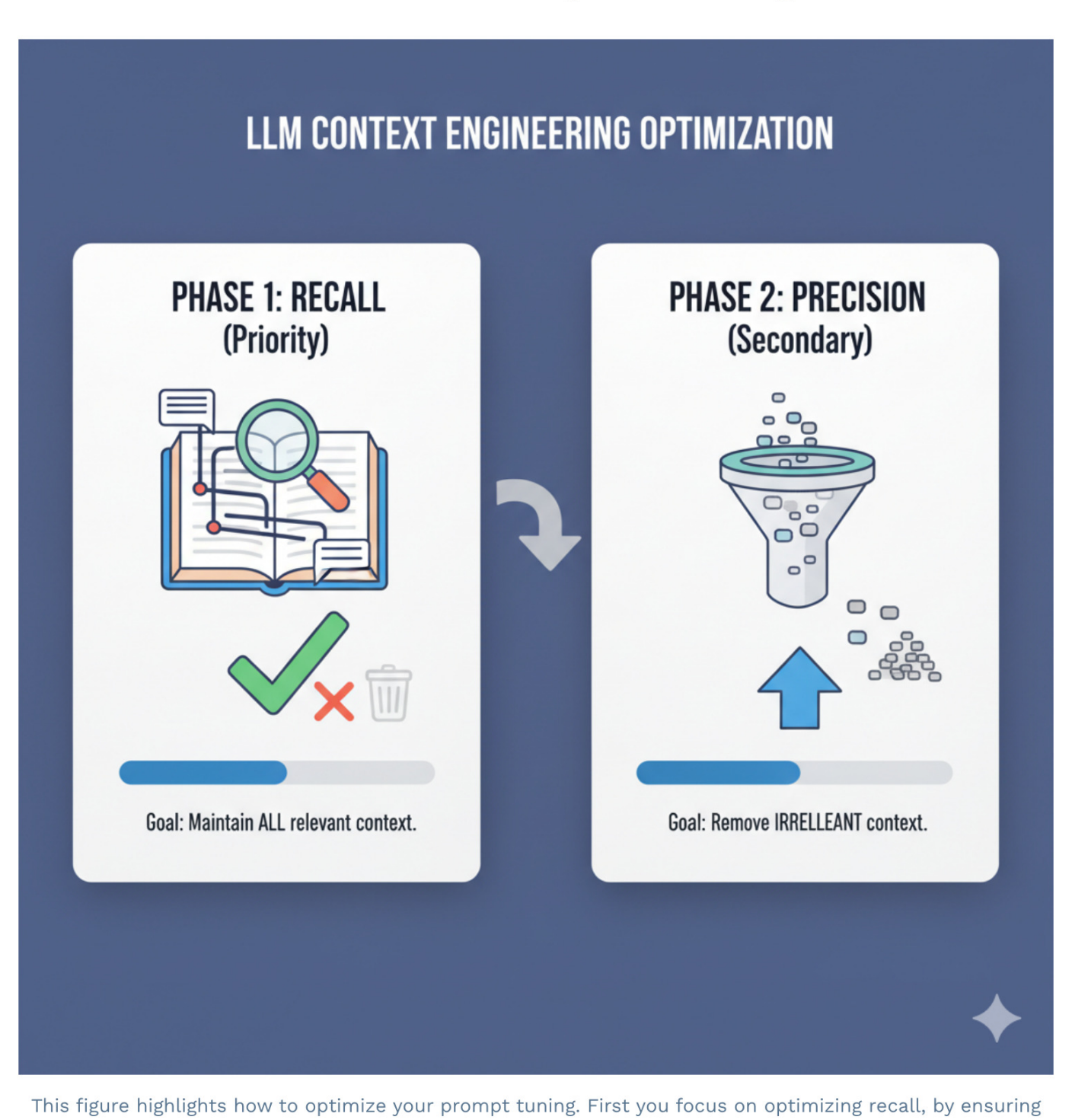
Determine if whole parts can be cut out

The first thing you should do when attempting to shorten the context is to find areas of the context that can be completely cut out.

For example, if the LLM might've previously fetched a document, used to solve a previous task, where you have the task results. This means the document is not relevant anymore and should be removed from the context. This might also occur if the LLM has fetched other information, for example via keyword search, and the LLM has itself summarized the output of the search. In this instance, you should remove the old output from the keyword search.

Simply removing such whole parts of the context can get you far in shortening the context. However, you need to keep in mind that removing context that can be relevant for later tasks can be detrimental to the agent's performance.

Thus, as Anthropic points out in their article on context engineering, you should first optimize for recall, where you ensure the LLM shortener never removes context that is relevant in the future. When you achieve almost perfect recall, you can start focusing on precision, where you remove more and more context that is not relevant anymore to solving the task at hand.


LLM CONTEXT ENGINEERING OPTIMIZATION
PHASE 1: RECALL
(Priority)
Goal: Maintain ALL relevant context.
PHASE 2: PRECISION
(Secondary)
Goal: Remove IRRELLEANT context.

This figure highlights how to optimize your prompt tuning. First you focus on optimizing recall, by ensuring all relevant context remain after summarization. Then in phase two, you start focusing on precision by removing less relevant context, from the memory of the agent. Image by Google Gemini.

Prompt-tuned shortening LLM

I also recommend creating a prompt-tuned shortening LLM. To do this, you first need to create a test set of contexts and the desired shortened context, given a task at hand. These examples should preferably be fetched from real user interactions with your agent.

Continuing, you can prompt optimize (or even fine-tune) the shortening LLM for the task of summarizing the LLM's context, to keep important parts of the context, while removing other parts of the context that are not relevant anymore.

Tools

One of the main points separating agents from one-off LLM calls is their use of tools. We typically provide agents with a series of tools they can use to increase the agent's ability to solve a task. Examples of such tools are:

- Perform a keyword search on a document corpus
- Fetch information about a user given their email
- A calculator to add numbers together

These tools simplify the problem the agent has to solve. The agent can perform a keyword search to fetch additional (often required) information, or it can use a calculator to add numbers together, which is much more consistent than adding numbers using next-token prediction.

Here are some techniques to keep in mind to ensure proper tool usage when providing tools in the agent's context:

- Well-described tools (can a human understand it?)
- Create specific tools
- Avoid bloating
- Only show relevant tools
- Informative error handling

Well-described agentic tools

The first, and probably most important note, is to have well-described tools in your system. The tools you define should have type annotations for all input parameters and a return type. It should also have a good function name and a description in the docstring. Below you can see an example of a poor tool definition, vs a good tool definition:

```python
# poor tool definition
def calculator(a, b):
  return a+b

# good tool definition
def add_numbers(a: float, b: float) -> float:
  """A function to add two numbers together. Should be used anytime you have to add two
     Takes in parameters:
       a: float
       b: float
     Returns
       float
  """
  return a+b
```

The second function in the code above is much easier for the agent to understand. Properly describing tools will make the agent much better at understanding when to use the tool, and when other approaches is better.

The go-to benchmark for a well-described tool is:

> *Can a human who has never seen the tools before, understand the tools, just from looking at the functions and their definitions?*

Specific tools

You should also try to keep your tools as specific as possible. When you define vague tools, it's difficult for the LLM to understand when to use the tool and to ensure the LLM uses the tool properly.

For example, instead of defining a generic tool for the agent to fetch information from a database, you should provide specific tools to extract specific info.

Bad tool:

- Fetch information from database
- Input
    - Columns to retrieve
    - Database index to find info by

Better tools:

- Fetch info about all users from the database (no input parameters)
- Get a sorted list of documents by date belonging to a given customer ID
- Get an aggregate list of all users and the actions they have taken in the last 24 hours

You can then define more specific tools when you see the need for them. This makes it easier for the agent to fetch relevant information into its context.

Avoid bloating

You should also avoid bloating at all costs. There are two main approaches to achieving this with functions:

1. Functions should return structured outputs, and optionally, only return a subset of results
2. Avoid irrelevant tools

For the first point, I'll again use the example of a keyword search. When performing a keyword search, for example, against AWS Elastic Search, you'll receive back a lot of information, sometimes not that structured.

```python
# bad function return
def keyword_search(search_term: str) -> str:
  # perform keyword search
  # results: {"id": ..., "content": ..., "createdAt": ..., ...}, {...}, {...}]
  return str(results)


# good function return
def _organize_keyword_output(results: list[dict], max_results: int) -> str:
  output_string = ""
  num_results = len(results)
  for i, res in enumerate(results[:max_results]): # max return max_results
    output_string += f"Document number {i}/{num_results}. ID: {res["id"]}, content: {re
  return output_string

def keyword_search(search_term: str, max_results: int) -> str:
  # perform keyword search
  # results: {"id": ..., "content": ..., "createdAt": ..., ...}, {...}, {...}]
  organized_results = _organize_keyword_output(results, max_results)
  return organized_results
```

In the bad example, we simply stringify the raw list of dicts returned from the keyword search. The better approach is to have a separate helper function to structure the results into a structured string.

You should also ensure the model can return only a subset of results, as shown with the *max_results* parameter. This helps the model a lot, especially with functions like keyword search, that can potentially return 100's of results, immediately filling up the LLM's context.

• • •

My second point was on avoiding irrelevant tools. You'll probably encounter situations where you have a lot of tools, many of which will only be relevant for the agent to use at specific steps. If you know a tool is not relevant for an agent at a given time, you should keep the tool out of the context.

Informative error handling

Informative error handling is critical when providing agents with tools. You need to help the agent understand what it's doing wrong. Usually, the raw error messages provided by Python are bloated and not that easy to understand.

Below is a good example of error handling in tools, where the agent is told what the error was and how to deal with it. For example, when encountering rate limit errors, we tell the agent to specifically sleep before trying again. This simplifies the problem a lot for the agent, as it doesn't have to reason itself that it has to sleep.

```python
def keyword_search(search_term: str) -> str:
  try:
    # keyword search
    results = ...
    return results
  except requests.exceptions.RateLimitError as e:
    return f"Rate limit error: {e}. You should run time.sleep(10) before retrying."
  except requests.exceptions.ConnectionError as e:
    return f"Connection error occurred: {e}. The network might be down, inform the user
  except requests.exceptions.HTTPError as e:
    return f"HTTP error occurred: {e}. The function failed with http error. This usually
  except Exception as e:
    return f"An unexpected error occurred: {e}"
```

You should have such error handling for all functions, keeping the following points in mind:

- Error messages should be informative of what happened
- If you know the fix (or potential fixes) for a specific error, inform the LLM how to act if the error occurs (for example: if a rate limit error, tell the model to run time.sleep())

Agentic context engineering going forward

In this article, I've covered three main topics: Specific context engineering tips, shortening the agents' context, and how to provide your agents with tools. These are all foundational topics you need to understand to build a good AI agent. There are also further topics that you should learn more about, such as the consideration of pre-computed information or inference-time information retrieval. I'll cover this topic in a future article. Agentic context engineering will continue to be a super relevant topic, and understanding how to handle the context of an agent is, and will be, fundamental to future AI agent developments.

👉 **My Free Resources**

🚀 10x Your Engineering with LLMs (Free 3-Day Email Course)

📚 Get my free Vision Language Models ebook

💻 My webinar on Vision Language Models

👉 **Find me on socials:**

📩 Subscribe to my newsletter

🧑‍💻 Get in touch

🔗 LinkedIn

🐦 X / Twitter

✍️ Medium

WRITTEN BY
Eivind Kjosbakken
Data scientist at Findable. Former CS Student at TU Delft and NTNU. I write articles about AI. Reach me at: https://www.linkedin.com/in/eivind-kjosbakken/

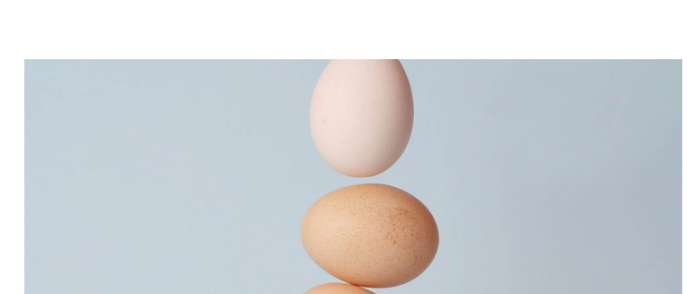
See all from Eivind Kjosbakken

Deep Dives AI Agent Context Engineering Llm Machine Learning

Share This Article

Towards Data Science is a community publication. Submit your insights to reach our global audience and earn through the TDS Author Payment Program.

Write for TDS

Related Articles